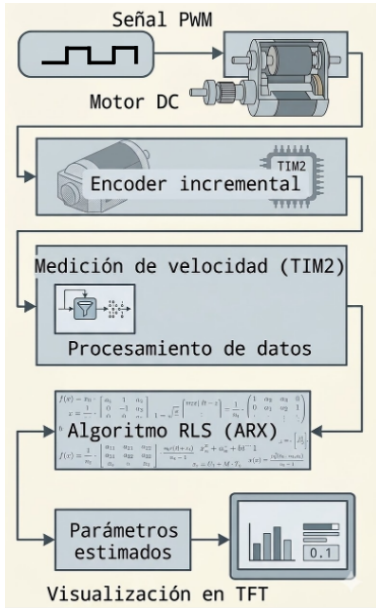


Módulo 1 - Configuración del reloj del sistema

Proyecto de identificación RLS/ARX en STM32F405RGT6

Archivos documentados: system_clock_driver.h, system_clock_driver.c, system_clock_if.h y system_clock_if.c.



Función del módulo. Inicializa el árbol de reloj del microcontrolador y ofrece retardos por software.

Por qué es el primer módulo. Todas las configuraciones posteriores dependen de una frecuencia del sistema conocida.

Frecuencia objetivo. SYSCLK = 168 MHz, AHB = 168 MHz, APB1 = 42 MHz, APB2 = 84 MHz.

Uso dentro del proyecto. En main.c se llama `SYSCLK_STM32.init()` antes de inicializar PWM, encoder, UART, TFT y temporizadores.

1. Papel del módulo dentro del firmware

En este proyecto el reloj del sistema es la base temporal de todo el firmware. Cuando el código configura el System Clock, define la velocidad a la que ejecuta el núcleo, la temporización de los buses AHB/APB y, de manera indirecta, la frecuencia real con la que trabajarán los periféricos como PWM, UART, SPI y temporizadores.

Por eso este módulo se ejecuta al inicio del programa. Si el reloj no se configura correctamente, las demás librerías pueden seguir compilando, pero sus cálculos de baudios, periodos, tiempos de muestreo o retardos dejarán de coincidir con el hardware real.

2. Relación entre archivos

Archivo	Responsabilidad
system_clock_driver.h	Declara las funciones de bajo nivel que configuran el reloj y generan retardos.
system_clock_driver.c	Implementa el acceso directo a registros RCC y FLASH.
system_clock_if.h	Define una interfaz abstracta basada en punteros a función.
system_clock_if.c	Construye la instancia constante SYSCLK_STM32 que enlaza interfaz y driver.

Fragmento 1. API directa expuesta por el driver

```
/* system_clock_driver.h */
void SystemClock_Config(void);
void Delay_loop(volatile uint32_t nCount);
void Delay_ms(volatile uint32_t ms);
```

3. Análisis detallado de SystemClock_Config()

La función `SystemClock_Config()` configura el STM32F405 para trabajar con un cristal externo de alta velocidad (HSE) y usar el PLL como fuente principal del sistema. El flujo seguido por el código es correcto y coherente con una inicialización bare-metal típica para llegar a 168 MHz.

Paso	Acción en el código	Efecto técnico
1	Activar HSE y esperar HSERDY.	Se habilita la fuente externa estable antes de usarla como entrada del PLL.
2	Configurar FLASH -> ACR con cachés, prefetch y 5 wait states.	La memoria Flash se adapta a la nueva frecuencia alta de CPU para evitar lecturas incorrectas.
3	Programar RCC -> PLLCFGR con M=8, N=336, P=2 y Q=7.	Se genera una frecuencia de PLL adecuada para SYSCLK y para el dominio USB de 48 MHz.
4	Activar PLL y esperar PLLRDY.	El oscilador sintetizado queda estable antes de usarse como reloj principal.
5	Ajustar prescalers AHB, APB1 y APB2.	Se respetan los límites máximos de los buses periféricos del microcontrolador.
6	Seleccionar PLL como fuente del sistema y esperar confirmación SWS_PLL.	El núcleo y la mayoría de los periféricos quedan sincronizados a la nueva configuración.
7	Desactivar HSI.	Se deja de usar el reloj interno por defecto para ahorrar consumo y evitar depender de una fuente que ya no es necesaria.

3.1 Cálculo de frecuencias

Magnitud	Expresión usada	Resultado
Entrada al VCO	$HSE / M = 8 \text{ MHz} / 8$	1 MHz
Salida del VCO	$1 \text{ MHz} \times 336$	336 MHz
PLLCLK / SYSCLK	$336 \text{ MHz} / 2$	168 MHz
USB / PLLQ	$336 \text{ MHz} / 7$	48 MHz
APB1	$168 \text{ MHz} / 4$	42 MHz
APB2	$168 \text{ MHz} / 2$	84 MHz

Fragmento 2. Núcleo de la configuración del reloj

```

void SystemClock_Config(void)
{
    RCC->CR |= RCC_CR_HSEON;
    while (!(RCC->CR & RCC_CR_HSERDY));

    FLASH->ACR = FLASH_ACR_ICEN | FLASH_ACR_DCEN |
        FLASH_ACR_PRFTEN | FLASH_ACR_LATENCY_5WS;

    RCC->PLLCFGR = (8 << RCC_PLLCFGR_PLLM_Pos)
        | (336 << RCC_PLLCFGR_PLLN_Pos)
        | (0 << RCC_PLLCFGR_PLLP_Pos)
        | (7 << RCC_PLLCFGR_PLLQ_Pos)
        | RCC_PLLCFGR_PLLSRC_HSE;

    RCC->CR |= RCC_CR_PLLON;
    while (!(RCC->CR & RCC_CR_PLLRDY));
}

```

Observación didáctica: en este proyecto la configuración se ejecuta una sola vez al inicio, por lo que el uso de `|=` sobre `RCC->CFGR` funciona sin problema práctico. Si en otro diseño se quisiera reconfigurar el reloj en tiempo de ejecución, sería más seguro limpiar previamente los bits del campo antes de escribir el nuevo prescaler.

4. Funciones de retardo por software

Además de configurar el reloj, el módulo ofrece dos rutinas de espera ocupada. Son útiles para secuencias simples de inicialización, por ejemplo durante el arranque del display TFT, pero no sustituyen a un temporizador hardware cuando se requiere precisión.

4.1 Delay_loop()

`Delay_loop()` recibe un contador y lo agota dentro de un doble ciclo que ejecuta `__NOP()`. Es un retardo bruto: bloquea la CPU y su duración depende de la frecuencia del reloj, del compilador y del nivel de optimización.

Fragmento 3. Implementación de Delay_loop()

```

void Delay_loop(volatile uint32_t nCount)
{
    uint8_t i = 0;
    while(nCount--)
    {
        for (i = 0; i < 100; i++)
        {
            __NOP();
        }
    }
}

```

4.2 Delay_ms()

`Delay_ms()` intenta aproximar un retardo en milisegundos mediante una calibración lineal. El código calcula internamente $nCount = 0.5286 + 274.9388 * ms$ y luego reutiliza el mismo patrón de espera ocupada. Esto sugiere que el autor ajustó la constante de forma empírica para que el resultado se pareciera a un milisegundo real bajo cierta frecuencia y ciertas opciones de compilación.

Punto importante para el alumno: esta función es válida para retardos sencillos de inicialización, pero no debe usarse como base de muestreo del algoritmo RLS ni para temporización crítica. Para eso es preferible usar un temporizador como TIM4 o SysTick.

Fragmento 4. Implementación de Delay_ms()

```
void Delay_ms(volatile uint32_t ms)
{
    uint32_t nCount = 0;
    nCount = 0.5286 + 274.9388 * ms;
    while(nCount--)
    {
        for (i = 0; i < 100; i++)
        {
            __NOP();
        }
    }
}
```

Ejemplo real en el proyecto. La rutina `reset_TFT()` usa `SYSCLK_STM32.delay_ms(12)` y `SYSCLK_STM32.delay_ms(122)` para respetar el tiempo de reset del controlador ILI9341.

5. Capa de interfaz: `system_clock_if`

El proyecto no llama siempre a las funciones del driver de manera directa. En su lugar define una estructura de interfaz con punteros a función. Esta decisión da un estilo más modular al firmware: la aplicación usa un objeto común (`SYSCLK_STM32`) y no depende del nombre exacto de cada implementación.

Fragmento 5. Definición de la interfaz

```
typedef void (*clk_delay_fn_t)(volatile uint32_t n);
typedef void (*clk_void_fn_t)(void);

typedef struct {
    clk_void_fn_t  init;
    clk_delay_fn_t delay_loop;
    clk_delay_fn_t delay_ms;
} system_clock_if_t;

extern const system_clock_if_t SYSCLK_STM32;
```

Fragmento 6. Inyección de la implementación concreta

```
const system_clock_if_t SYSCLK_STM32 = {
    .init      = SystemClock_Config,
    .delay_loop = Delay_loop,
    .delay_ms  = Delay_ms
};
```

6. Secuencia de uso en la aplicación

En `main.c` la primera inicialización funcional del firmware es precisamente el reloj. Después de limpiar banderas de fallo y habilitar la FPU, el programa ejecuta `SYSCLK_STM32.init()`. Solo entonces continúa con pantalla, encoder, PWM, UART y GPIO.

Fragmento 7. Uso de la interfaz desde `main.c`

```
int main(void)
{
    clear_fault_flags();
    FPU_Enable();

    SYSCLK_STM32.init();
    ini_pantalla();
    quad_STM32.init();
    PWM_STM32.motor.init(PWM_TARGET_HZ);
    usart_STM32.init_115200();
    ...
}
```

Después del reloj...	Por qué depende del reloj
PWM del motor y del backlight TFT	El cálculo de periodo y prescaler parte de una frecuencia base conocida.
USART3 a 115200 bps	El baudio correcto depende del PCLK del bus donde vive el periférico.
TIM2 encoder y TIM4 temporización	La medición de velocidad y el tiempo de muestreo requieren una base temporal consistente.
SPI1 y display ILI9341	La velocidad de transmisión y los tiempos de reset/arranque dependen del reloj de bus y de los retardos.

7. Ideas clave para el estudiante

- El reloj del sistema no es un detalle menor: define la escala temporal del resto del firmware.
- La secuencia HSE -> FLASH -> PLL -> prescalers -> SYSCLK debe respetarse para evitar comportamientos inestables.
- Los retardos por software son simples pero bloqueantes; son útiles para arranque, no para control o muestreo preciso.
- La capa `if` no cambia el hardware, pero sí mejora la organización del código y desacopla la aplicación del driver directo.
- Cuando un periférico no funciona, muchas veces el error real no está en el periférico, sino en un reloj mal configurado.

8. Conclusión del módulo

El bloque `system_clock` es la base de inicialización del proyecto. Su función principal es llevar al STM32F405 desde el estado de arranque por defecto hasta una configuración de 168 MHz estable, y además proporcionar esperas simples usadas por otras librerías. Desde el punto de vista didáctico, este módulo enseña dos cosas al mismo tiempo: cómo se programa el árbol de reloj a nivel de registros y cómo se puede encapsular esa funcionalidad mediante una interfaz más limpia para la aplicación.